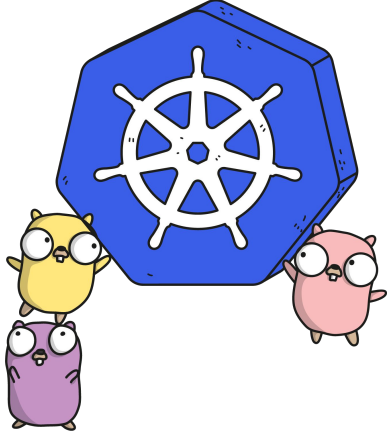


Kubernetes' Architecture Fundamentals

Lucas Käldström - luxas labs

18th October 2017 - London

Image credit: @ashleymcnamara

$ whoami

Lucas Käldström, Upper Secondary School Student, just turned 18

CNCF Ambassador and **Certified Kubernetes Administrator**

Speaker at **KubeCon Berlin** 2017 and now at **KubeCon Austin** later this year

Kubernetes Maintainer since April 2016, active in the community for +2 years

Driving **luxas labs** which currently performs contracting for Weaveworks

A guy that has never attended a computing class

Image credit: Dan Kohn

EVERYONE'S EXCITED ABOUT

KUBERNETES

Most importantly: What does “Kubernetes” mean?

Kubernetes
= Greek for “pilot” or
“helmsman of a ship”

What is Kubernetes?

= A Production-Grade Container Orchestration System

Google-grown, based on **Borg** and **Omega**, systems that run inside of Google right now and are proven to work at Google for over 10 years.
Google spawns *2 billion containers per week* with these systems.

Created by three Google employees initially during the **summer of 2014**;
grew exponentially and became the first project to get donated to the CNCF.

Hit the first production-grade version (**v1.0.1) after a year**.

Have continually released a *new minor version every three months* since v1.2.0 in March 2016. **v1.8.0** was just released **28th September 2017**.

So what does Kubernetes *actually* do?

One thing: ***Abstract away the underlying hardware***. Abstract away the concept **Node.**

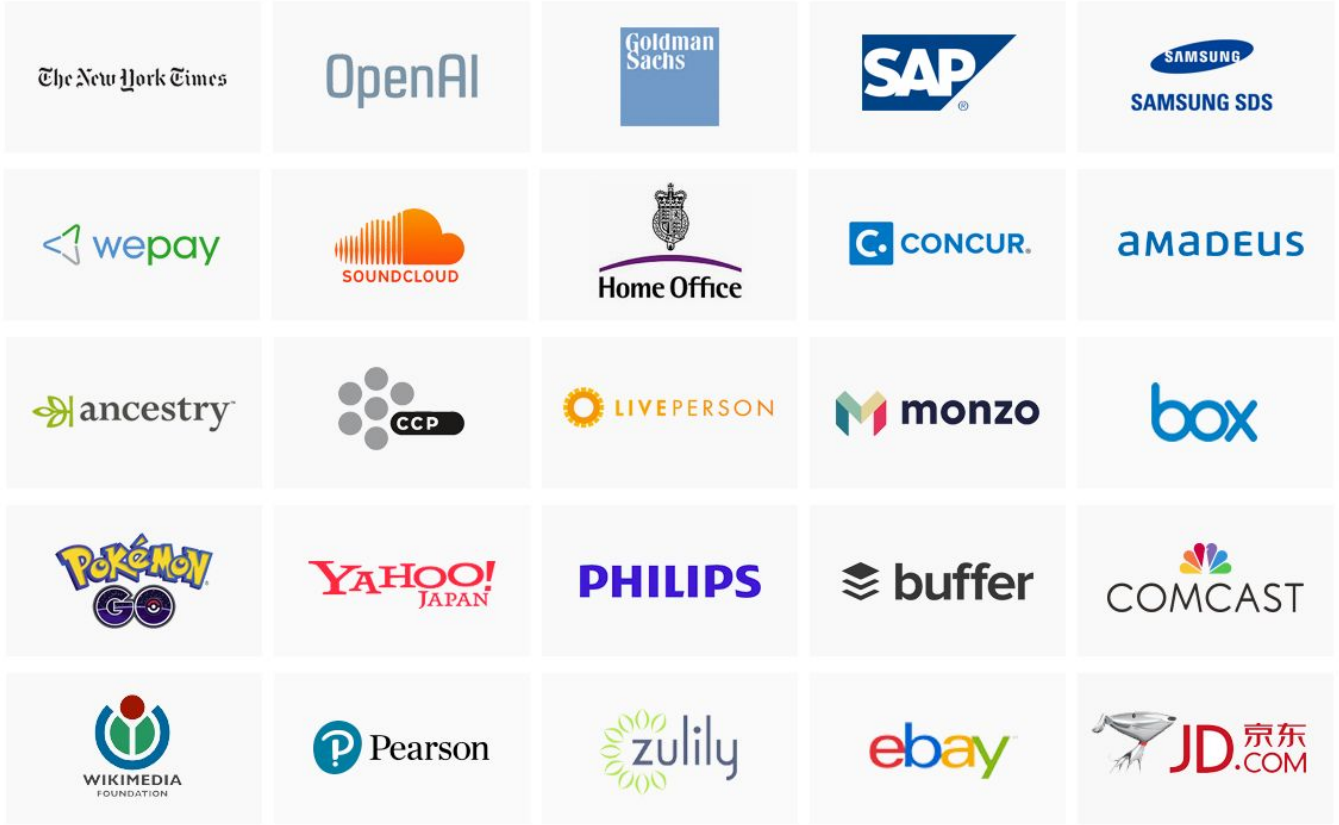
Principle: Manage your applications like **Cattle** (generic, bulk operations) instead of like **Pets** (every operation is customized with care and love for the individual)

Kubernetes is the ***Linux for distributed systems***.

In the same manner Linux (an OS) abstracts away the hardware differences (with different CPU types, etc.), Kubernetes abstracts away the fact that you have 5 000 nodes in the node pool and provides consistent UX and operation methods for apps

You (the admin) declares the **desired state**, Kubernetes' main task is to ***make the desired state the actual state*.**

A couple of Kubernetes users



Stats about the Kubernetes project

22 000+
users on Slack

60 000+
commits
the latest year

11 000+
edX course enrolls
in less than 3 months

2 300+
unique authors

5 700+
Kubernetes jobs

32 000+
opened
Pull Requests
the latest year

18 000+
opened issues
the latest year

~23 PRs
merges/day
in the core repo

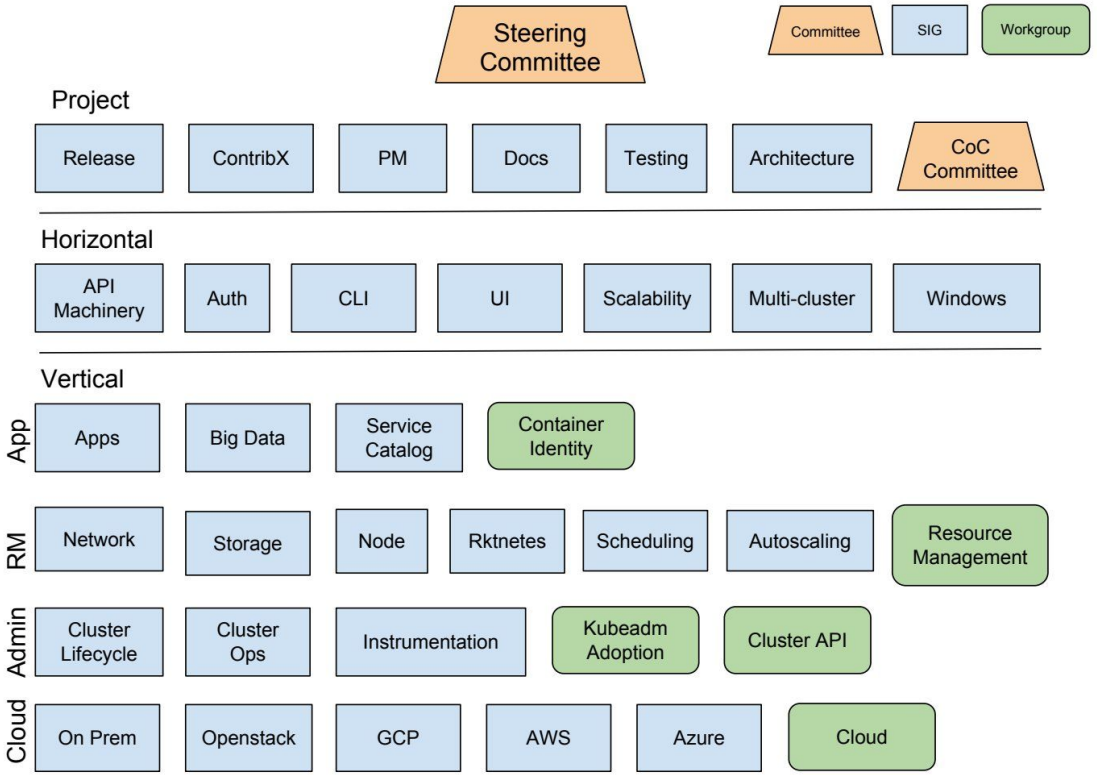
25 000+
Kubernetes
professionals

Last updated: 1.10.2017

[Kubernetes is one of the fastest moving open source projects in history]

[Source 1] [Source 2] [Source 3] [Source 4]

Everything is done in SIG (Special Interest Groups)



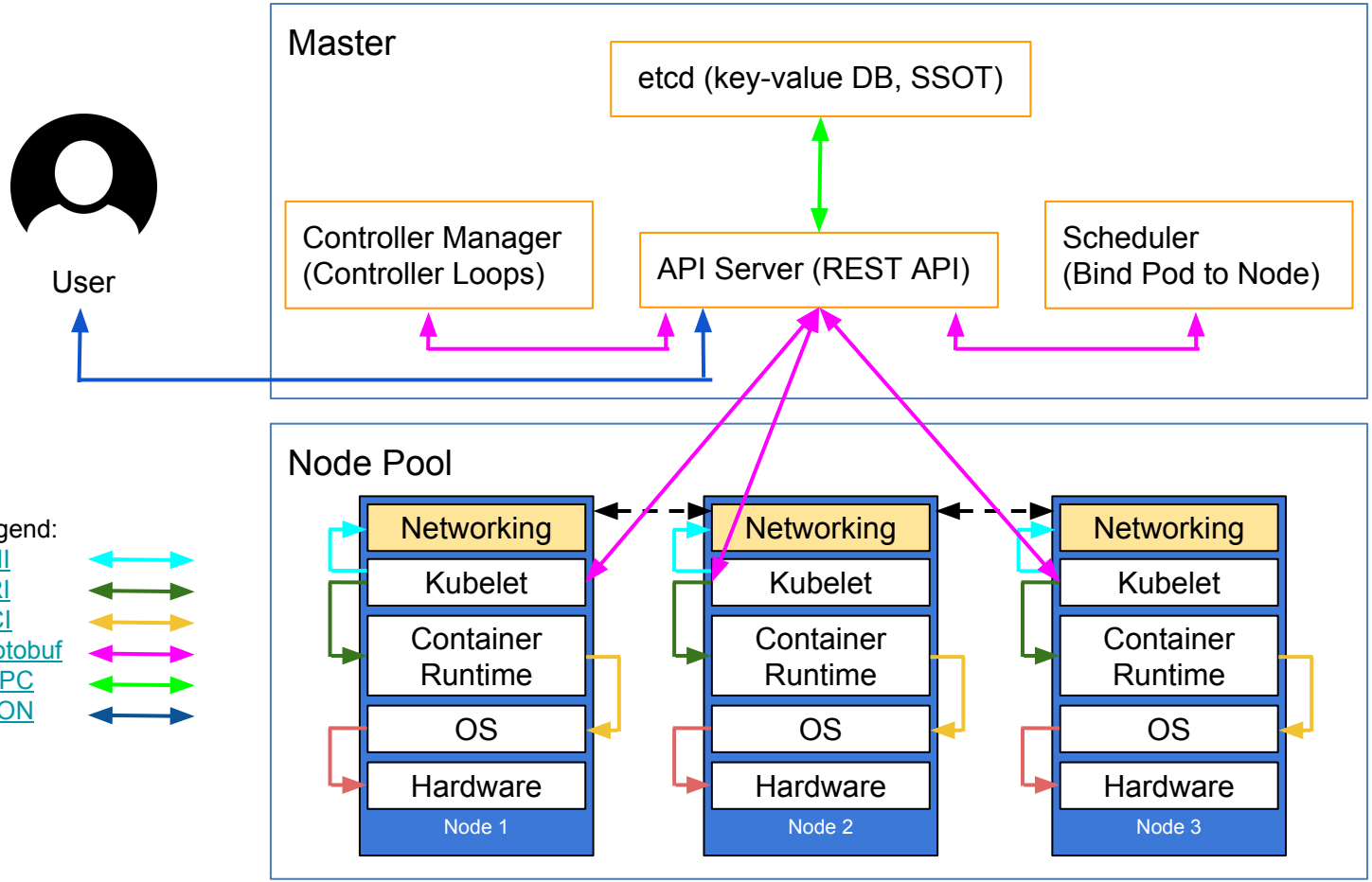
Special Interest Groups manage Kubernetes' various components and features.

All code in the Kubernetes Github organization should be owned by one or more SIGs; with directory-level granularity.

SIGs have regular (often weekly) video meetings where the attendees discuss design decisions, new features, bugs, testing, onboarding or whatever else that is relevant to the group. Attending these meetings is the best way to get to know the project

Image source

Kubernetes high-level component architecture



What are Kubernetes' core concepts?

Pod: The basic and atomically schedulable building block of Kubernetes, represents *a single instance of an application in Kubernetes.* Each Pod has it's own, uniquely assigned and **internal IP**. Pods are **mortal**.

Deployment: Includes a Pod template and a replicas field. Kubernetes will make sure the actual state (amount of replicas, Pod template) always matches the desired state. When you update a Deployment it will perform a "rolling update".

Service: Selects Pods by a matching label selector and provides a stable, **immortal** way to talk to your application by using the **internal IP** or DNS name.

Namespace: A logical isolation method, most resources are namespace-scoped. You can then group logically similar workloads and enforce various policies.

Ok, show me what a Kubernetes manifest looks like!

```yaml
apiVersion: apps/v1beta2
kind: Deployment
metadata:
  labels:
    app: webapp
    role: frontend
  name: web-frontend
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: webapp
        role: frontend
    spec:
      containers:
      - image: nginx:1.13.1
        name: nginx
        ports:
        - containerPort: 80
          name: http
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-frontend
spec:
  selector:
    app: webapp
    role: frontend
  ports:
  - port: 80
    targetPort: 80
```

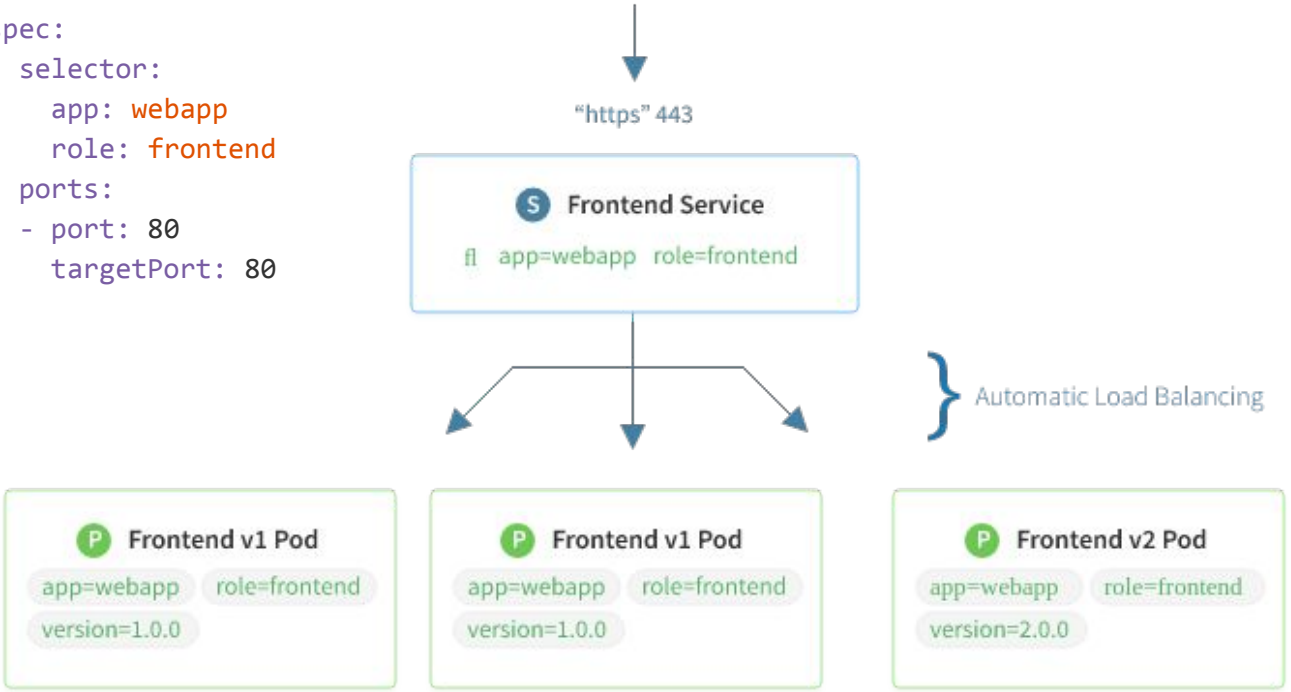
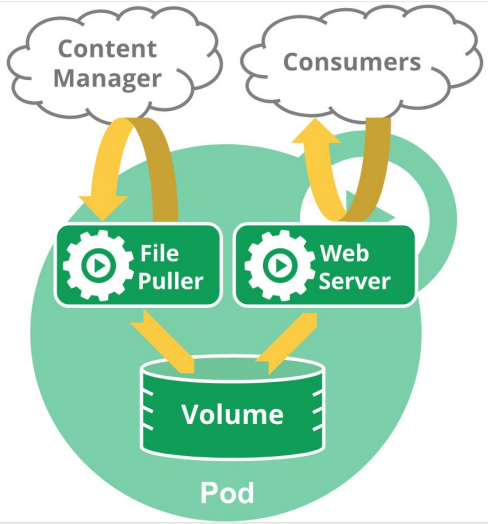


Image source

Each Pod can include 1 to *n* containers in its spec

This was one of the key takeaways from Google's experience with Borg (8.2)

You can use this pattern in specific instances where your containers are tightly coupled. For instance, you might have a container that acts as a web server for files in a shared volume, and a separate "sidecar" container that updates those files from a remote source, as in the following diagram



Pods provide two kinds of shared resources for their constituent containers: **networking** and **storage**.

Every container in a Pod shares the network namespace, including the IP address and network ports. Containers inside a Pod can communicate with one another using *localhost*. A Pod can specify a set of shared storage volumes. All containers in the Pod can access the shared volumes. **Volumes** also allow persistent data in a Pod to survive in case one of the containers within needs to be restarted.

TODO source

The heart of the cluster -- the API Server

Is at its core a stateless **REST web server** serving the different **Kubernetes APIs**

Is backed by some kind of distributed database system (currently only **etcd**)

Is the **policy layer** of Kubernetes providing filtered access to the backing key-value database

Handles (pluggable and programmable) **Authentication and Authorization**, request validation, modification or rejection with **Admission Controllers** and **API Versioning** that makes it possible to render the same backing data in different consumable API versions.

In addition to the normal CRUD operations supported, it also supports **WATCH**.

The hardworking Controller Manager

Runs up to 26 different controller loops (in v1.8.0) that ensure the **actual state** of the cluster equals the **desired state**.

Solely talks to the API server; each controller loop has its own credential.

Leader election is supported, so you can run many instances for HA.

The business logic lives here; each controller loop has its own, important responsibility for keeping the system running properly.

Example: The user creates a **Deployment**, and the *deployment controller* creates a **ReplicaSet**. The *replicaset controller* creates or deletes **Pods** to make the amount of **Pods** match the desired amount of replicas.

The Pod-binding Scheduler

Binds an unbound Pod to a Node

Solely talks to the API server; it has its own credential.

Takes a lot of different information from the cluster into account for the scheduling, and schedules the **Pod** to the best matching Node.

Leader election is supported, so you can run many instances for HA.

The scheduler implementation is pluggable -- you can write and use your own scheduler easily.

Want to know more about the default scheduler? Read this excellent post

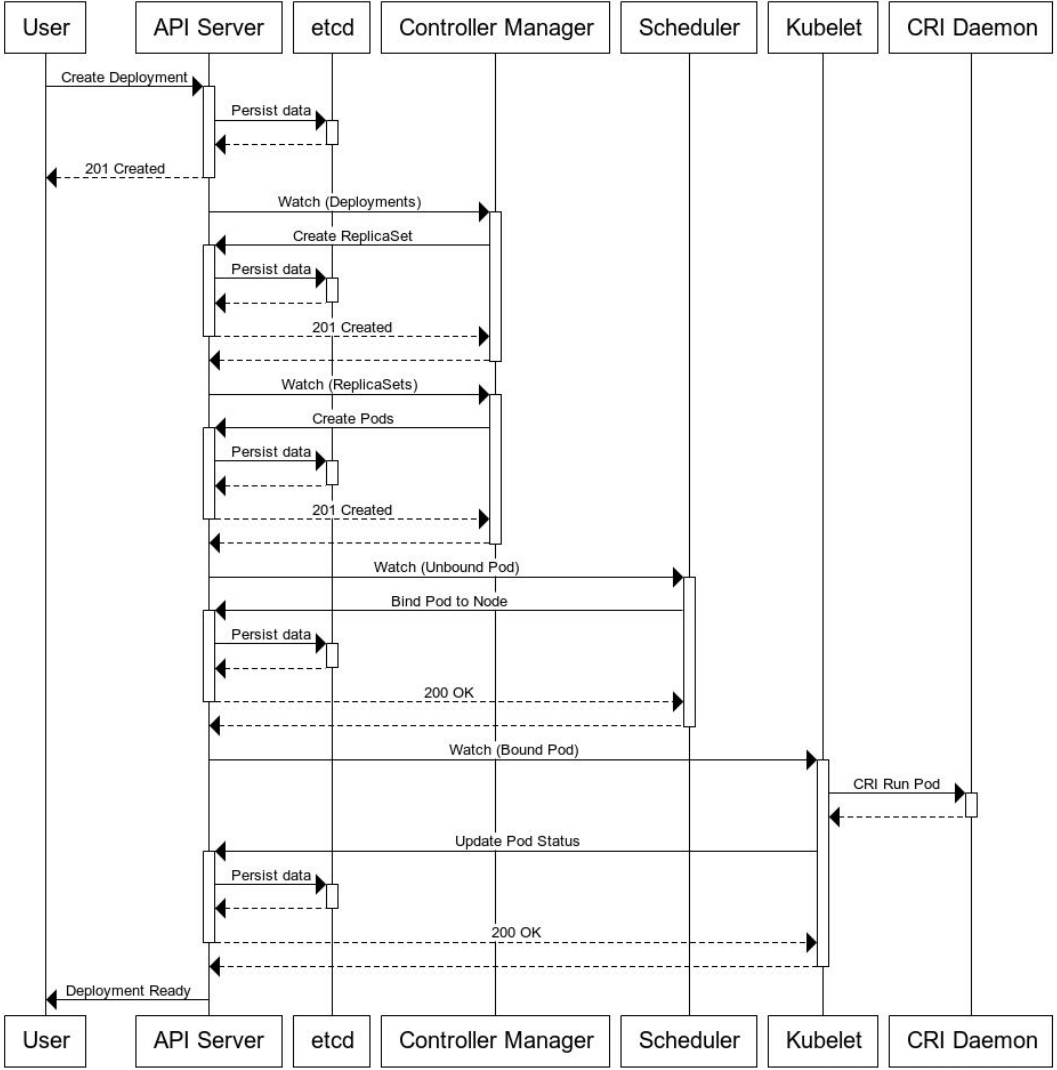
The Pod runner -- the Kubelet

The kubelet is the *Node agent* that actually makes **Pods** run on **Nodes**.

The Kubelet **watches Pods** bound to it and fetches other linked resources (like **ConfigMaps** and **Secrets**) *on demand* when running a Pod.

When the Kubelet notices a new Pod bound to it, it delegates the actual running of the Pod to a container runtime of choice (e.g. Docker). The container runtime needs to support (or have a shim for) the **Container Runtime Interface (CRI).**

Each Pod has its *unique, mortal and internal* ***IP address.*** In order to achieve this, the Kubelet exec's out to a 3rd-party **Container Network Interface (CNI)** plugin that sets up the network in a manner that satisfies Kubernetes' needs.



What happens when you create a Deployment?

Here's a sequence diagram of what happens when you create a Deployment.

Note that the API server (and the backing key-value DB) are involved in virtually any operation in the cluster.

The "business logic" components (the controller-manager and scheduler) acts when something interesting happens.

Finally, the Kubelet receives the message it should run a Pod; and delegates that task to a container runtime (like docker) via a daemon shim implementing the Container Runtime Interface (CRI).

The networking ninjas -- kube-proxy & CNI plugins

As discussed earlier, each Pod has its *unique, mortal and internal* ***Pod IP address***.

Any third-party provider that implements the CNI spec (like Weave Net), can be used for creating the network between Nodes and Pods in the cluster.

However, we need a stable way to talk to a group of Pods -- enter **Services**.

The Service matches a set of Pods and has a internal **Service IP address**.

The **kube-proxy** daemon (deployed as a **DaemonSet**) watches **Pods** and **Services** in the cluster and makes the Service IP *forward traffic* to **the set of Pod IPs**.

The kube-dns addon resolves `$service.$namespace(.svc.cluster.local)` to the right **Service IP**. This way you can talk to other apps by knowing only the Service

Kubernetes clients -- kubectl & the Dashboard

kubectl is the command-line client for the Kubernetes API

kubectl is a very powerful tool that you can use for inspecting, debugging and modifying the cluster state.

The Kubernetes Dashboard project aims to make it easier to visualize and consume the cluster state for those who aren't familiar with the command line. The dashboard is installed as a Deployment + Service on top of Kubernetes.

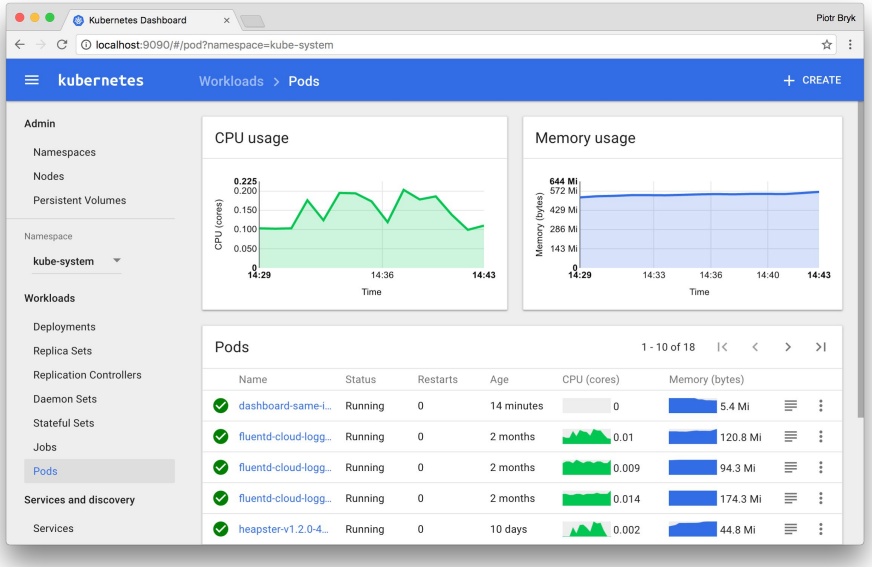


Image source

Kubernetes extension points

CNI (Container Network Interface) - write your own network plugin for the cluster

CRI (Container Runtime Interface) - write your own container runtime for Kubernetes

CSI (Container Storage Interface) - write your own persistent storage plugin

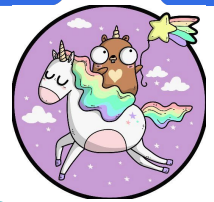
Pluggable cloud providers - write your own extension that integrates with your cloud

CustomResourceDefinitions - create your own Kubernetes Resources easily

API Aggregation - proxy a new API group from the core API server to your extension

Initializers & External Admission Webhooks - write your own Admission Controllers

How do I kick the tires with Kubernetes?



Play with Kubernetes right away in your browser!

Create a single-node cluster on your laptop or workstation with minikube

Create a real cluster with only a couple of commands with kubeadm

Create a production-ready cluster on AWS with kops

Create a Kubernetes cluster on GCE with GKE (Google Container Engine)

kubicorn is a Kubernetes installer project which has gained some traction

Create a cluster with kubeadm

1. Provision a Linux machine with Ubuntu, Debian, RHEL, CentOS or Fedora
2. Install kubeadm:

```
curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | apt-key add -
cat <<EOF >/etc/apt/sources.list.d/kubernetes.list
deb http://apt.kubernetes.io/ kubernetes-xenial main
EOF
apt-get update && apt-get install -y kubeadm docker.io
```

3. Make kubeadm set up a master node for you:

```
kubeadm init
```

4. Install a Pod Network solution from a third-party provider:

```
kubectl apply -f https://git.io/weave-kube-1.6
```

5. Repeat step 1 & 2 on an other node and join the cluster:

```
kubeadm join --token <token> <master-ip>:6443
```

What now?

Follow the Kubernetes blog, YouTube channel & Twitter feed

Do as 11 000+ others and take the free edX "Introduction to Kubernetes" course

Join 22 500+ others in the Kubernetes Slack: http://slack.k8s.io

Prep for and take the Certified Kubernetes Administrator exam

Join a Special Interest Group and attend the weekly meetings

Kick the tires with Kubernetes on your machines with minikube or kubeadm

Check out the weekly Kubernetes Community Meeting at Zoom

Read the in-depth analysis of the Kubernetes ecosystem ebook by The New Stack

Thank you!

@luxas on Github
@kubernetesonarm on Twitter
lucas@luxaslabs.com

Various excellent blog posts

Core Kubernetes: Jazz Improv over Orchestration - Joe Beda, 30th of May 2017

Kubernetes deep dive: API Server, Part 1 - Michael Hausenblas & Stefan Schimanski, 28th April 2017

Kubernetes deep dive: API Server, Part 2 - Michael & Stefan, 21st July 2017

Kubernetes deep dive: API Server, Part 3 - Michael & Stefan, 15th August 2017

Reasons Kubernetes is cool - Julia Evans, 5th October 2017

How Kubernetes certificate authorities work - Julia Evans, 5th August 2017

Operating a Kubernetes network - Julia Evans, 10th October 2017